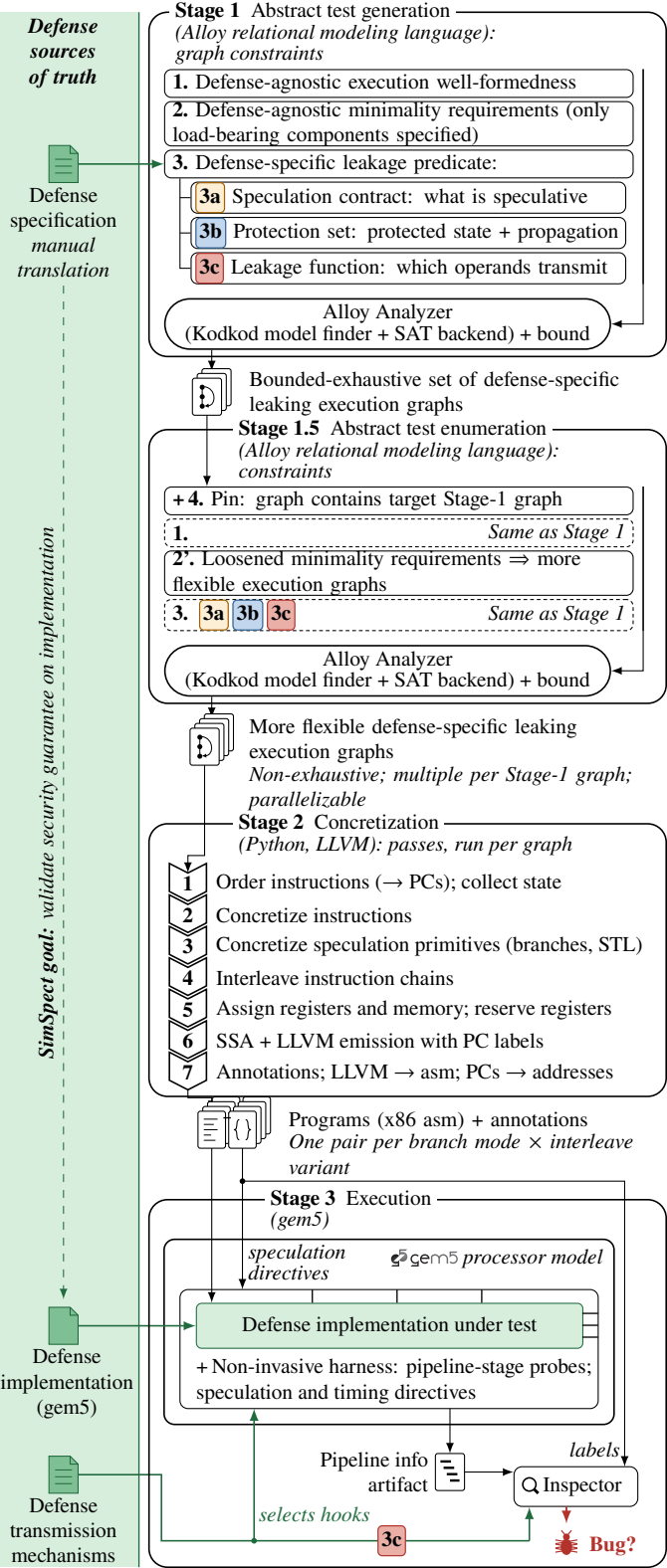

Defense sources of truth
Stage 1 Abstract test generation
(Alloy relational modeling language): graph constraints
1. Defense-agnostic execution well-formedness
2. Defense-agnostic minimality requirements (only load-bearing components specified)
3. Defense-specific leakage predicate:
3a Speculation contract: what is speculative
3b Protection set: protected state + propagation
3c Leakage function: which operands transmit
Alloy Analyzer (Kodkod model finder + SAT backend) + bound
Defense specification manual translation
Bounded-exhaustive set of defense-specific leaking execution graphs
Stage 1.5 Abstract test enumeration
(Alloy relational modeling language): constraints
+4. Pin: graph contains target Stage-1 graph
1. Same as Stage 1
2'. Loosened minimality requirements ⇒ more flexible execution graphs
3. 3a 3b 3c Same as Stage 1
Alloy Analyzer (Kodkod model finder + SAT backend) + bound
More flexible defense-specific leaking execution graphs
Non-exhaustive; multiple per Stage-1 graph; parallelizable
Stage 2 Concretization
(Python, LLVM): passes, run per graph
1 Order instructions (→ PCs); collect state
2 Concretize instructions
3 Concretize speculation primitives (branches, STL)
4 Interleave instruction chains
5 Assign registers and memory; reserve registers
6 SSA + LLVM emission with PC labels
7 Annotations; LLVM → asm; PCs → addresses
Programs (x86 asm) + annotations
One pair per branch mode × interleave variant
Stage 3 Execution
(gem5)
speculation directives
gem5 processor model
Defense implementation under test
+ Non-invasive harness: pipeline-stage probes; speculation and timing directives
Defense implementation (gem5)
Pipeline info artifact
labels
Inspector
selects hooks
3c
Bug?
Defense transmission mechanisms
SimSpect goal: validate security guarantee on implementation